

Reporting Bugs

Yegor Bugayenko

Lecture #2 out of 8
80 minutes

The slidedeck was presented by the author in this [YouTube Video](#).

All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as web sites. Copyright belongs to their respected authors.



JOEL SPOLSKY

“Every good bug report needs exactly three things: steps to reproduce, what you expected to see, and what you saw instead.”

— Joel Spolsky. *Painless Bug Tracking*.
<https://jttu.net/spolsky2000bugs>. [Online; accessed 07-02-2024]. Nov. 2000



NICOLAS BETTENBURG

“Well-written bug reports are likely to get more attention among developers than poorly written ones... Steps to reproduce and stack traces are most useful in bug reports. The most severe problems encountered by developers are errors in steps to reproduce, incomplete information, and wrong observed behavior.”

— Nicolas Bettenburg et al. “What Makes a Good Bug Report?” In: *Proceedings of the 16th International Symposium on Foundations of Software Engineering*. 2008, pp. 308–318. DOI: [10.1145/1453101.1453146](https://doi.org/10.1145/1453101.1453146)



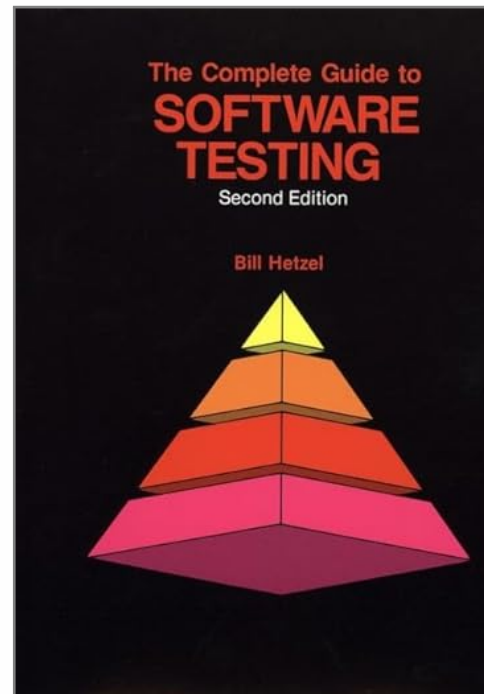
TOMMASO DAL SASSO

“The elements considered to be harder to provide [while reporting bugs] are the entity (e.g., class, file) that likely contains the defect, the steps to reproduce the failure, and a test case showing the defect.”

— Tommaso Dal Sasso, Andrea Mocci, and Michele Lanza. “What Makes a Satisficing Bug Report?” In: *Proceedings of the International Conference on Software Quality, Reliability and Security (QRS)*. IEEE. 2016, pp. 164–174. DOI: [10.1109/QRS.2016.28](https://doi.org/10.1109/QRS.2016.28)

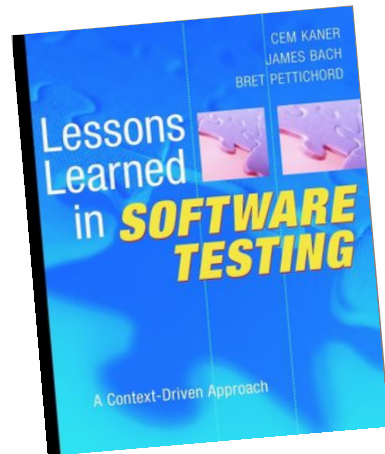


1. Expect any program to have an unlimited number of bugs



“We can never be certain that a testing system is correct. These theoretical limit tells us that there will never be a way to be sure we have a perfect understanding of what a program is supposed to do (the expected or required results) and that any testing system we might construct will always have some possibility of failing. In short, we cannot achieve 100 percent confidence no matter how much time and energy we put into it!”

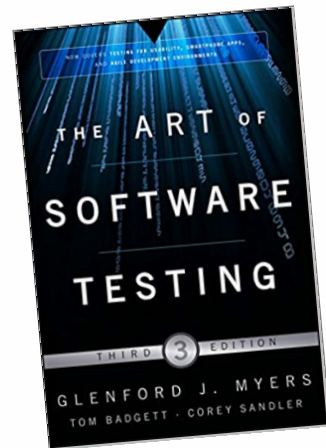
— Bill Hetzel. *The Complete Guide to Software Testing*. Wiley, 1993



CEM KANER

“Different people have different expectations of a product, but if a legitimate stakeholder is disappointed by the product and feels that it’s less valuable because of something that it does or doesn’t do, she should write that up as a bug report.”

— Cem Kaner, James Bach, and Bret Pettichord. *Lessons Learned in Software Testing: A Context-Driven Approach*. Wiley, 2002. DOI: [10.5555/559964](https://doi.org/10.5555/559964)



MYERS, GLENFORD J.

“You cannot test a program to guarantee that it is error free... It is impractical, often impossible, to find all the errors in a program.”

— Glenford J. Myers et al. *The Art of Software Testing*. 2nd ed. Wiley, 2012.
DOI: [10.5555/2161638](https://doi.org/10.5555/2161638)



2. Don't ask questions or suggest features,
report bugs instead

All of These Are Bugs:

- Lack of functionality
- Lack of tests
- Lack of documentation
- Suboptimal implementation
- Design inconsistency
- Naming is weird
- Unstable test

More about it by Bugayenko [7].



REIS CHRISTIAN

“Though it commonly has a pejorative connotation, in the Mozilla Project the term **bug** is used to refer to any field request for modification in the software, be it an actual defect, an enhancement, or a change in functionality (“bug-driven development”).”

— Christian Robottom Reis and Renata Pontin de Mattos Fortes. *An Overview of the Software Engineering Process and Tools in the Mozilla Project*. Open Source Software Development Workshop, 2002



KIM HERZIG

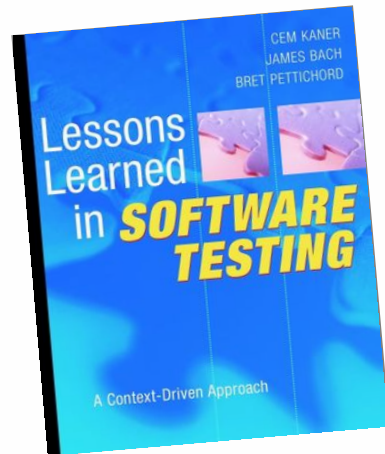
“In a manual examination of more than 7,000 issue reports from five open-source projects, we found 33.8% of all bug reports to be misclassified threatening bug prediction models, confusing bugs and features. On average, 39% of files marked as defective actually never had a bug.”

— Kim Herzig, Sascha Just, and Andreas Zeller. “It’s Not a Bug, It’s a Feature: How Misclassification Impacts Bug Prediction”. In: *Proceedings of the 35th International Conference on Software Engineering*. IEEE. 2013, pp. 392–401. DOI: [10.1109/ICSE.2013.6606585](https://doi.org/10.1109/ICSE.2013.6606585)



3. Reward yourself for each reported bug [5].

4. Report strictly one problem per ticket



JAMES BACH

“If you write a report that has multiple bugs on it, some of them will probably never be fixed.”

— Cem Kaner, James Bach, and Bret Pettichord. *Lessons Learned in Software Testing: A Context-Driven Approach*. Wiley, 2002. DOI: [10.5555/559964](https://doi.org/10.5555/559964)



JOHN ANVIK

“People play different roles as they interact with reports in a bug repository. The person who submits the report is the reporter or the submitter of the report. The triager is the person who decides if the report is meaningful and who assigns responsibility of the report to a developer. The one that resolves the report is the resolver. A person that contributes a fix for a bug is called a contributor.”

— John Anvik, Lyndon Hiew, and Gail C. Murphy. “Who Should Fix This Bug?” In: *Proceedings of the 28th International Conference on Software Engineering*. 2006, pp. 361–370. DOI: [10.1145/1134285.1134336](https://doi.org/10.1145/1134285.1134336)

5. Exaggerate for effect [8].

Do This, While Reporting Bugs:

- Stay cool
- Exaggerate
- Victimize yourself
- Push them
- Show efforts
- Look engaged
- Look altruistic
- Aggregate (not!)

More about it by Bugayenko [8].



6. Simplify until it's impossible to simplify any further [4].

Bugs Occam's Razor

```
1 Here is my code:
2
3 a := 7
4 a := a + 5 - 3
5 a := a / 3
6 print a
7
8 It doesn't work as expected.
9 It prints 4, but it should
10 print 3.
```

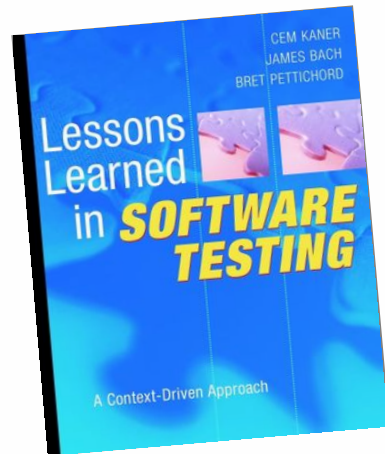
```
1 Here is my code:
2
3 a := 7
4 a := a - 3
5 print a
6
7 It doesn't work as expected.
8 It prints 7, but it should
9 print 4.
```

Most probably, the subtracting operator doesn't do anything — this is the bug.

7. Be a prosecutor, not an advocate [6].



8. Don't give up until some changes are made to the code base [6].



BRET PETTICHORD

“No bug should be marked as closed unless it has been reviewed and closed by a tester.”

— Cem Kaner, James Bach, and Bret Pettichord. *Lessons Learned in Software Testing: A Context-Driven Approach*. Wiley, 2002. DOI: [10.5555/559964](https://doi.org/10.5555/559964)



9. Prefer a disabled test in lieu of a bug report [3].

Why not?

```
1 // @todo #42 This test is disabled
2 // because the fibo() doesn't work
3 // correctly with this input,
  returning
4 // 17711 instead of 28657. Fix it.
5 #[test]
6 #[ignore]
7 fn calculates_23rd_fibonacci_number
  () {
8   let x = fibo(23);
9   assert_eq!(28657, x);
10 }
```

“Such a PR serves as both a bug report (this is what the text of the puzzle will be turned into, once the PR is merged) and a test that reproduces the problem. It will be more than welcome by the repository maintenance team. This kind of PR saves the time they would spend creating a unit test. Also, it saves your time for creating a bug report, as it will be created automatically by the puzzles discovery tool.” [3]

Repositories With a Lot of Bugs (9 Feb 2024)

Github Repository	Issues
<u>mozilla bugzilla</u>	1,8M+
<u>gitlab-org/gitlab</u>	172,462
<u>flutter/flutter</u>	79,386
<u>kubernetes/kubernetes</u>	42,627
<u>tensorflow/tensorflow</u>	36,776
<u>moby/moby</u>	19,367

All repositories are open source.

References

- [1] John Anvik, Lyndon Hiew, and Gail C. Murphy. "Who Should Fix This Bug?" In: *Proceedings of the 28th International Conference on Software Engineering*. 2006, pp. 361–370. DOI: [10.1145/1134285.1134336](https://doi.org/10.1145/1134285.1134336).
- [2] Nicolas Bettenburg et al. "What Makes a Good Bug Report?" In: *Proceedings of the 16th International Symposium on Foundations of Software Engineering*. 2008, pp. 308–318. DOI: [10.1145/1453101.1453146](https://doi.org/10.1145/1453101.1453146).
- [3] Yegor Bugayenko. *A Disabled Test in Lieu of a Bug Report*. <https://www.yegor256.com/230725.html>. [Online; accessed 07-02-2024]. July 2023.
- [4] Yegor Bugayenko. *Bugs Occam's Razor*. <https://www.yegor256.com/220329.html>. [Online; accessed 07-02-2024]. Mar. 2022.
- [5] Yegor Bugayenko. *Either Bugs or Pull Requests ... or You Are Out*. <https://www.yegor256.com/180724.html>. [Online; accessed 07-02-2024]. July 2018.
- [6] Yegor Bugayenko. *Five Principles of Bug Tracking*. <https://www.yegor256.com/141124.html>. [Online; accessed 07-02-2024]. Nov. 2014.
- [7] Yegor Bugayenko. *More Bugs, Please*. <https://www.yegor256.com/180206.html>. [Online; accessed 07-02-2024]. Feb. 2018.
- [8] Yegor Bugayenko. *The Right Way to Report a Bug*. <https://www.yegor256.com/180424.html>. [Online; accessed 08-02-2024]. Apr. 2018.
- [9] Tommaso Dal Sasso, Andrea Mocci, and Michele Lanza. "What Makes a Satisficing Bug Report?" In: *Proceedings of the International Conference on Software Quality, Reliability and Security (QRS)*. IEEE. 2016, pp. 164–174. DOI: [10.1109/QRS.2016.28](https://doi.org/10.1109/QRS.2016.28).
- [10] Kim Herzig, Sascha Just, and Andreas Zeller. "It's Not a Bug, It's a Feature: How Misclassification Impacts Bug Prediction". In: *Proceedings of the 35th International Conference on Software Engineering*. IEEE. 2013, pp. 392–401. DOI: [10.1109/ICSE.2013.6606585](https://doi.org/10.1109/ICSE.2013.6606585).
- [11] Bill Hetzel. *The Complete Guide to Software Testing*. Wiley, 1993.
- [12] Cem Kaner, James Bach, and Bret Pettichord. *Lessons Learned in Software Testing: A Context-Driven Approach*. Wiley, 2002. DOI: [10.5555/559964](https://doi.org/10.5555/559964).
- [13] Glenford J. Myers et al. *The Art of Software Testing*. 2nd ed. Wiley, 2012. DOI: [10.5555/2161638](https://doi.org/10.5555/2161638).
- [14] Christian Robottom Reis and Renata Pontin de Mattos Fortes. *An Overview of the Software Engineering Process and Tools in the Mozilla Project*. Open Source Software Development Workshop, 2002.
- [15] Joel Spolsky. *Painless Bug Tracking*. <https://jttu.net/spolsky2000bugs>. [Online; accessed 07-02-2024]. Nov. 2000.